Министерство науки и высшего образования Российской Федерации
Федеральное государственное автономное образовательное учреждение
высшего образования
«Московский государственный технический университет
имени Н.Э. Баумана
(национальный исследовательский университет)»
(МГТУ им. Н.Э. Баумана)

ФАКУЛЬТЕТ ИНФОРМАТИКА И СИСТЕМЫ УПРАВЛЕНИЯ
КАФЕДРА СИСТЕМЫ ОБРАБОТКИ ИНФОРМАЦИИ И УПРАВЛЕНИЯ

ЛАБОРАТОРНАЯ РАБОТА № 1

«Основные конструкции языка Python»

ДИСЦИПЛИНА: «Парадигмы и конструкции языков программирования»

Выполнил: студент гр. ИУ5-31Б _____ (Арифулов Р.Х.)
(Подпись) (Ф.И.О.)

Проверил: _____ (Гапанюк Ю.Е.)
(Подпись) (Ф.И.О.)

Дата сдачи (защиты):

Результаты сдачи (защиты):

- Балльная оценка:
- Оценка:

2026 г.

1 Цель работы

Изучение основных конструкций языка Python.

2 Задание

Разработать программы для решения биквадратного уравнения $Ax^4 + Bx^2 + C = 0$.

Программы должны быть разработаны в виде консольного приложения на языке Python. Необходимо разработать три программы:

– с применением процедурной парадигмы (использование функций, оператор def);

– с применением объектно-ориентированной парадигмы (использование классов);

– с применением функциональной парадигмы (сопоставление с образцом).

Каждая программа осуществляет ввод с клавиатуры коэффициентов A, B, C, вычисляет дискриминант и действительные корни уравнения (в зависимости от дискриминанта).

Коэффициенты A, B, C могут быть заданы в виде параметров командной строки. Если они не заданы, то вводятся с клавиатуры.

Если коэффициент A, B, C введен или задан в командной строке некорректно, то необходимо проигнорировать некорректное значение и вводить коэффициент повторно, пока коэффициент не будет введен корректно. Корректно заданный коэффициент — это коэффициент, значение которого может быть без ошибок преобразовано в действительное число.

3 Текст программы

Для решения биквадратного уравнения делается замена $t = x^2$, после чего решается квадратное уравнение $At^2 + Bt + C = 0$. Для каждого неотрицательного корня t находятся корни $x = \pm\sqrt{t}$. Если $A = 0$, уравнение сводится к $Bx^2 + C = 0$.

3.1 Процедурная парадигма

Файл procedural.py:

```
import sys
import math


def get_coef(index, prompt):
    # сначала пробуем взять коэффициент из командной строки
    if index < len(sys.argv):
        try:
            return float(sys.argv[index])
        except ValueError:
            print('Некорректное значение в командной строке:',
sys.argv[index])

    while True:
        text = input(prompt)
        try:
            return float(text)
        except ValueError:
            print('Нужно ввести число, попробуйте еще раз')


def get_roots(a, b, c):
    result = []
    if a == 0:
        # уравнение вырождается в b*x^2 + c = 0
        if b == 0:
            return result
        t_list = [-c / b]
    else:
        d = b * b - 4 * a * c
        print('Дискриминант D =', d)
        if d < 0:
            return result
        if d == 0:
            t_list = [-b / (2 * a)]
        else:
            sqrt_d = math.sqrt(d)
            t_list = [(-b + sqrt_d) / (2 * a), (-b - sqrt_d) / (2 * a)]

    for t in t_list:
        if t > 0:
            x = math.sqrt(t)
            result.append(x)
            result.append(-x)
        elif t == 0:
            result.append(0.0)

    return sorted(set(result))


def main():
    a = get_coef(1, 'Введите коэффициент A: ')
    b = get_coef(2, 'Введите коэффициент B: ')
    c = get_coef(3, 'Введите коэффициент C: ')

    if a == 0 and b == 0:
        print('Коэффициенты A и B равны нулю, это не уравнение')
        return
```

```
    roots = get_roots(a, b, c)
    if len(roots) == 0:
        print('Действительных корней нет')
    else:
        print('Количество корней:', len(roots))
        for i, root in enumerate(roots, 1):
            print(f'x{i} = {root}')


if __name__ == '__main__':
    main()

# Пример запуска
# python procedural.py 1 -5 4
```

3.2 Объектно-ориентированная парадигма

Файл oop.py:

```
import sys
import math


class BiquadraticEquation:

    def __init__(self):
        self.a = 0.0
        self.b = 0.0
        self.c = 0.0
        self.d = None
        self.roots = []

    def read_coef(self, index, prompt):
        if index < len(sys.argv):
            try:
                return float(sys.argv[index])
            except ValueError:
                print('Некорректное значение в командной строке:',
sys.argv[index])

        while True:
            text = input(prompt)
            try:
                return float(text)
            except ValueError:
                print('Нужно ввести число, попробуйте еще раз')

    def read_all(self):
        self.a = self.read_coef(1, 'Введите коэффициент A: ')
        self.b = self.read_coef(2, 'Введите коэффициент B: ')
        self.c = self.read_coef(3, 'Введите коэффициент C: ')

    def solve_square(self):
        # корни для t = x^2
        if self.a == 0:
            return [-self.c / self.b]
        self.d = self.b * self.b - 4 * self.a * self.c
        if self.d < 0:
            return []
        if self.d == 0:
            return [-self.b / (2 * self.a)]
        sqrt_d = math.sqrt(self.d)
```

```
        return [(-self.b + sqrt_d) / (2 * self.a), (-self.b - sqrt_d) / (2 *
self.a)]

    def calculate(self):
        result = set()
        for t in self.solve_square():
            if t > 0:
                result.add(math.sqrt(t))
                result.add(-math.sqrt(t))
            elif t == 0:
                result.add(0.0)
        self.roots = sorted(result)

    def print_roots(self):
        if self.d is not None:
            print('Дискриминант D =', self.d)
        if len(self.roots) == 0:
            print('Действительных корней нет')
        else:
            print('Количество корней:', len(self.roots))
            for i, root in enumerate(self.roots, 1):
                print(f'x{i} = {root}')


def main():
    eq = BiquadraticEquation()
    eq.read_all()
    if eq.a == 0 and eq.b == 0:
        print('Коэффициенты A и B равны нулю, это не уравнение')
        return
    eq.calculate()
    eq.print_roots()


if __name__ == '__main__':
    main()
```

3.3 Функциональная парадигма

Файл functional.py:

```
import sys
import math


def to_float(text):
    try:
        return float(text)
    except ValueError:
        return None


def ask_coef(prompt):
    value = to_float(input(prompt))
    match value:
        case None:
            print('Нужно ввести число, попробуйте еще раз')
            return ask_coef(prompt)
        case _:
            return value


def get_coef(index, prompt):
```

```
    match sys.argv[index:index + 1]:
        case [text] if to_float(text) is not None:
            return to_float(text)
        case [text]:
            print('Некорректное значение в командной строке:', text)
            return ask_coef(prompt)
        case _:
            return ask_coef(prompt)


def square_roots(a, b, c):
    match (a, b):
        case (0, 0):
            return []
        case (0, _):
            return [-c / b]

    d = b * b - 4 * a * c
    print('Дискриминант D =', d)
    match d:
        case d if d < 0:
            return []
        case 0:
            return [-b / (2 * a)]
        case _:
            return [(-b + math.sqrt(d)) / (2 * a), (-b - math.sqrt(d)) / (2 *
a)]


def roots_from_t(t):
    match t:
        case t if t > 0:
            return [math.sqrt(t), -math.sqrt(t)]
        case 0:
            return [0.0]
        case _:
            return []


def get_roots(a, b, c):
    all_roots = [x for t in square_roots(a, b, c) for x in roots_from_t(t)]
    return sorted(set(all_roots))


def print_roots(roots):
    match roots:
        case []:
            print('Действительных корней нет')
        case _:
            print('Количество корней:', len(roots))
            for i, root in enumerate(roots, 1):
                print(f'x{i} = {root}')


def main():
    a = get_coef(1, 'Введите коэффициент A: ')
    b = get_coef(2, 'Введите коэффициент B: ')
    c = get_coef(3, 'Введите коэффициент C: ')

    match (a, b):
        case (0, 0):
            print('Коэффициенты A и B равны нулю, это не уравнение')
        case _:
            print_roots(get_roots(a, b, c))
```

```
if __name__ == '__main__':
    main()
```

4 Экранные формы с примерами выполнения программы

На рисунке 1 приведен запуск процедурной программы с коэффициентами, заданными в командной строке. Уравнение имеет четыре корня.

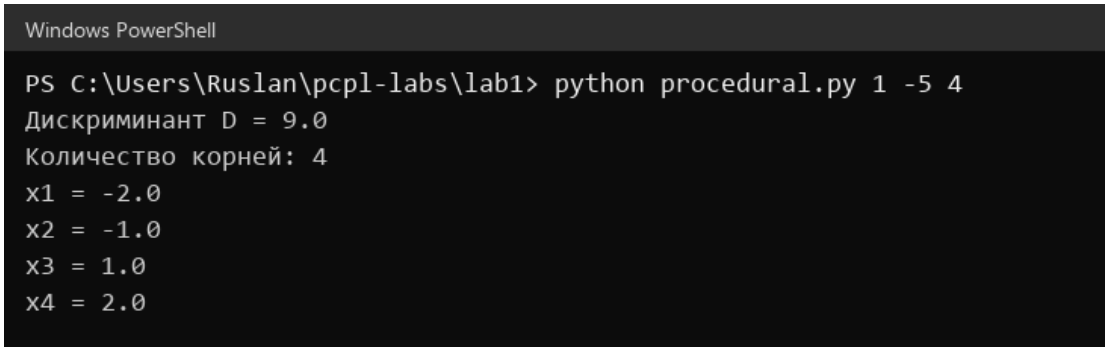


Рисунок 1 — Процедурная программа, параметры командной строки

На рисунке 2 показан ввод коэффициентов с клавиатуры, при некорректном вводе программа запрашивает коэффициент повторно.

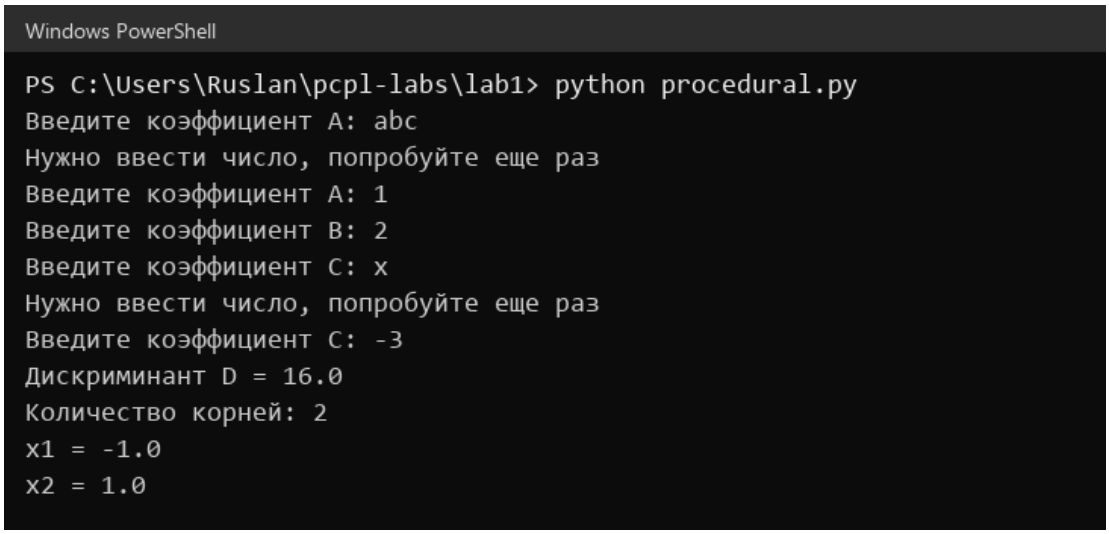


Рисунок 2 — Процедурная программа, ввод с клавиатуры

На рисунке 3 приведен пример работы объектно-ориентированной программы для уравнения без действительных корней.

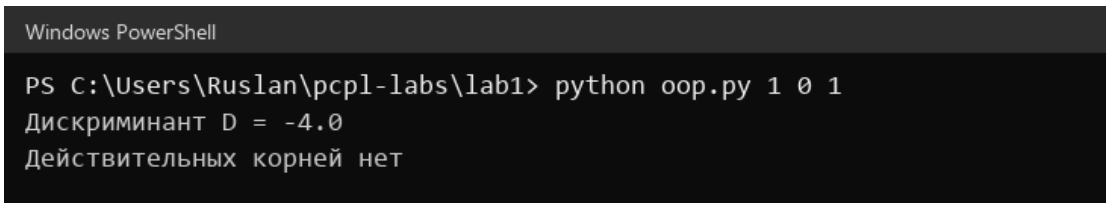


Рисунок 3 — Объектно-ориентированная программа, корней нет

На рисунке 4 первый коэффициент в командной строке задан некорректно, поэтому он вводится с клавиатуры.

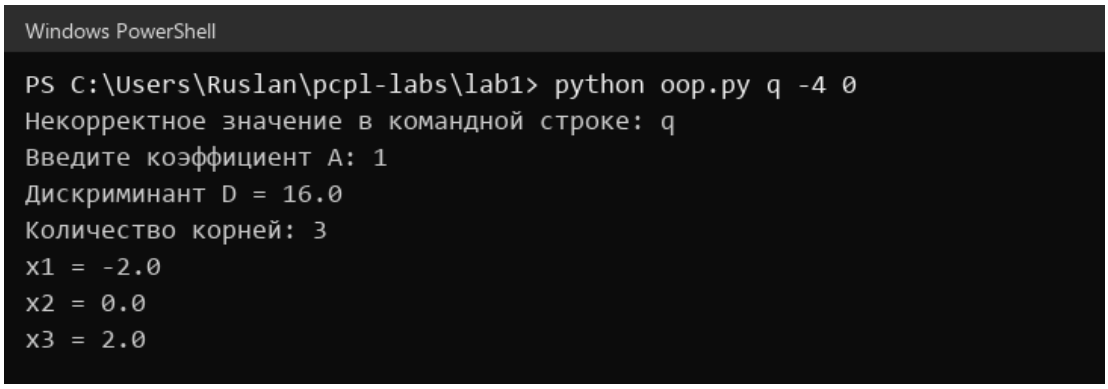


Рисунок 4 — Объектно-ориентированная программа, некорректный параметр

На рисунках 5–7 приведены примеры работы программы, разработанной с применением функциональной парадигмы.

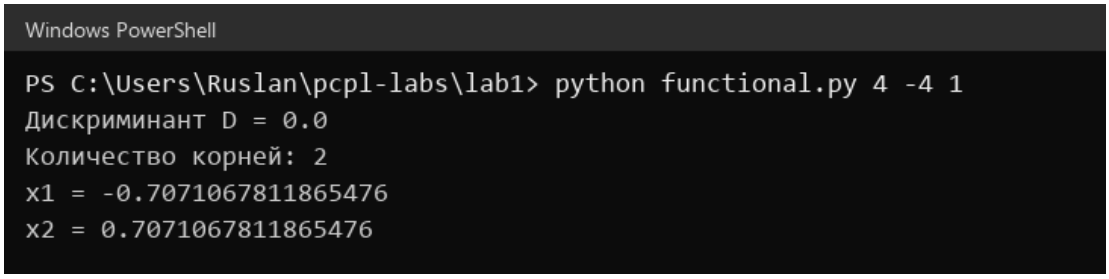


Рисунок 5 — Функциональная программа, дискриминант равен нулю

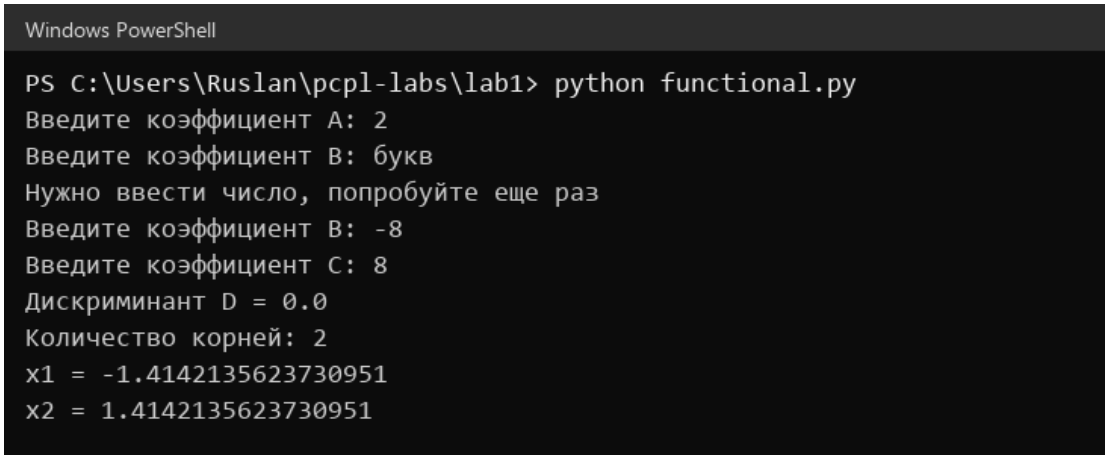


Рисунок 6 — Функциональная программа, ввод с клавиатуры

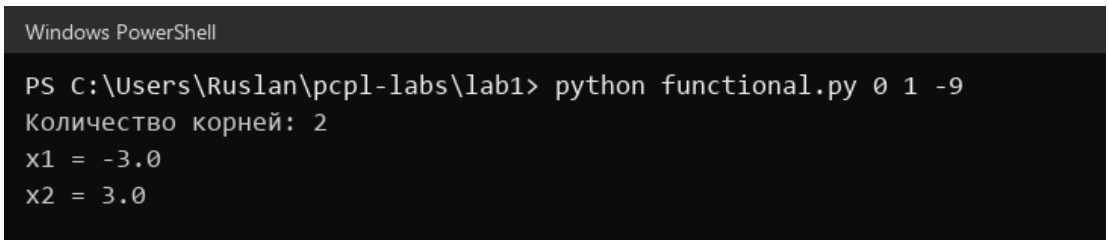


Рисунок 7 — Функциональная программа, коэффициент A равен нулю